

APLICACIONES MOVILES

LABORATORIO 2

Una pantalla que vive y muere

Módulo 2 — Desarrollo nativo · Guía autónoma · Tiempo estimado: 90–120 minutos

El problema de hoy: tu aplicación va a ser interrumpida, tapada, rotada y asesinada — hoy vas a verlo pasar en vivo, y vas a entender por qué las pantallas "vuelven en blanco".

Qué necesitás antes de empezar: el proyecto **HolaYo** del Laboratorio 1, funcionando (si no lo tenés, ese laboratorio es prerequisite — este construye encima). Emulador o teléfono, indistinto; para la Etapa 5 el emulador es más cómodo.

Teoría que acompaña: la anatomía de una aplicación Android — los componentes, los Intents, la pila de retroceso, los estados del ciclo de vida y la muerte por memoria. Este laboratorio es esa teoría, vista con tus propios ojos: cada cosa que el texto afirma, acá la vas a presenciar.

El destino

Al terminar, HolaYo va a tener una segunda pantalla —tu sala de experimentos— a la que se llega con un botón y que recibe un dato de la primera. Las dos pantallas van a *contar su propia vida* en el registro del sistema: vas a ver el entrelazado exacto de la navegación, vas a completar una tabla de experimentos prediciendo y verificando, vas a presenciar el momento preciso en que un giro de pantalla destruye tu estado, vas a matar el proceso con tus manos — y vas a salir con una pregunta incómoda que este laboratorio, a propósito, **no responde**.

Etapa 1 — La segunda pantalla

Una aplicación real tiene varias pantallas, y navegar entre ellas es apilar componentes. Esta etapa crea la segunda y la conecta con la primera — con un dato viajando en el medio.

1.1 — Creá la Activity. En el panel del proyecto, click derecho sobre tu paquete (`com.example.holayo`) → **New** → **Activity** → **Empty Views Activity**. Nombrala `SegundaActivity` y dejá tildado "Generate a Layout File". Antes de tocar nada, andá a mirar **AndroidManifest.xml**: Android Studio registró la Activity nueva por vos — ahí está, declarada ante el sistema, como manda la anatomía: un componente no existe hasta que el manifiesto lo dice.

✓ **Verificación:** `SegundaActivity.kt` y `activity_segunda.xml` creados, y la etiqueta `<activity android:name=".SegundaActivity" />` visible en el manifiesto.

1.2 — Dale contenido mínimo. Abrió `res/layout/activity_segunda.xml` y reemplazá su contenido:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:gravity="center"
    android:padding="24dp">

    <TextView
        android:id="@+id/tvBienvenida"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:textSize="20sp"
        android:textStyle="bold" />
```

```
</LinearLayout>
```

✓ *Verificación: la vista Design muestra la columna con el texto (vacío) centrado.*

1.3 — El viaje: un Intent explícito con equipaje. Agregá un botón al layout de la primera pantalla (en `activity_main.xml`, debajo del botón existente):

```
<Button
    android:id="@+id/btnIrSegunda"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginTop="12dp"
    android:text="@string/btn_ir_segunda" />
```

Su texto, en `strings.xml`: `<string name="btn_ir_segunda">Ir a la sala de experimentos</string>`. Y en `MainActivity.kt`, dentro de `onCreate`, el viaje:

```
val btnIrSegunda = findViewById<Button>(R.id.btnIrSegunda)
btnIrSegunda.setOnClickListener {
    // Intent EXPLÍCITO: nombra al destino por su clase.
    val intent = Intent(this, SegundaActivity::class.java)
    // El equipaje: un dato que viaja con el mensaje.
    intent.putExtra("nombre", perfil.apodo ?: perfil.nombre)
    startActivity(intent)
}
```

(Android Studio te va a pedir el import de `android.content.Intent` — aceptá la sugerencia con `Alt+Enter`.) Fijate qué es lo que *no* hicimos: no creamos la pantalla nueva, no la mostramos, no la administramos. **Le pedimos al sistema** que la active — el Intent es un mensaje al sistema, y es el sistema quien crea la Activity, la apila sobre la actual y la pone en escena. Vos pediste; él orquestó.

1.4 — Del otro lado: recibir el equipaje. En `SegundaActivity.kt`:

```
class SegundaActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_segunda)

        val nombre = intent.getStringExtra("nombre") ?: "misterioso visitante"
        findViewById<TextView>(R.id.tvBienvenida).text =
            "Sala de experimentos de $nombre"
    }
}
```

Ese `?: "misterioso visitante"` no es decoración: el extra *podría no venir* (si alguien activara esta pantalla sin el dato), y Kotlin te obliga al plan B — la nulabilidad del Laboratorio 1, ahora protegiendo la comunicación entre componentes.

Ejecutá.

✓ *Verificación: tocás el botón, la segunda pantalla aparece con tu nombre en el saludo; el botón/gesto de "atrás" te devuelve a la primera, tal como la dejaste. Acabás de usar la pila de retroceso — aunque todavía no la viste trabajar. Eso es exactamente lo que viene.*

Problemas frecuentes de la Etapa 1

- **"Unresolved reference: SegundaActivity"**: la clase quedó en otro paquete (mirá la primera línea `package ...` de ambos archivos — deben coincidir) o falta el import.
- **La app crashea al tocar el botón** con `ActivityNotFoundException`: la Activity no está en el manifiesto (raro si la creaste con el asistente, seguro si la creaste a mano). Agregá la etiqueta `<activity>` vos mismo.
- **El saludo dice "misterioso visitante"**: el nombre del extra no coincide — la clave `"nombre"` tiene que ser idéntica en `putExtra` y en `getStringExtra`, letra por letra.

Etapa 2 — Las pantallas hablan: el ciclo de vida, instrumentado

El sistema le avisa a cada Activity cada transición de su vida. Esos avisos son invisibles — hasta que los hacés hablar.

2.1 — Conocé el Logcat. Abrí la ventana **Logcat** (barra inferior de Android Studio). Es la consola donde el dispositivo cuenta todo lo que pasa — y es ruidosa: miles de líneas de todo el sistema. La herramienta es el **filtro**: en su barra de búsqueda escribí `tag:VIDA` — desde ahora, solo vas a ver los mensajes que nosotros marquemos con esa etiqueta. Logcat filtrado es, junto con el depurador, la herramienta de diagnóstico que vas a usar toda la materia.

✓ *Verificación: Logcat abierto, filtro `tag:VIDA` puesto, listado vacío (todavía no emitimos nada).*

2.2 — Instrumentá la primera pantalla. En `MainActivity.kt`, sobreescribí los métodos del ciclo de vida — agregalos completos al final de la clase, dentro de sus llaves:

```
override fun onStart() { super.onStart(); Log.d("VIDA", "Main → onStart") }
override fun onResume() { super.onResume(); Log.d("VIDA", "Main → onResume") }
}
override fun onPause() { super.onPause(); Log.d("VIDA", "Main → onPause") }
override fun onStop() { super.onStop(); Log.d("VIDA", "Main → onStop") }
override fun onDestroy() { super.onDestroy(); Log.d("VIDA", "Main →
onDestroy") }
```

Y la primera línea dentro del `onCreate` existente (después del `super.onCreate(...)`): `Log.d("VIDA", "Main → onCreate")`. (Import: `android.util.Log`.) Cada método es un aviso del sistema: "estás por ser visible", "sos interactiva", "te están tapando", "ya no se te ve", "te destruyo". Nosotros solo dejamos constancia escrita de cada aviso.

2.3 — Instrumentá la segunda igual — mismos seis métodos en `SegundaActivity.kt`, con la etiqueta `"VIDA"` y el prefijo `"Segunda → "`.

✓ *Verificación: ejecutá la app y mirá Logcat: al abrir aparece, en orden, `Main → onCreate`, `Main → onStart`, `Main → onResume`. La secuencia de nacimiento completa — el texto de la teoría, impreso por tu propia pantalla.*

2.4 — El entrelazado: navegá mirando el log. Ahora el experimento central de la etapa. Limpiá el Logcat (ícono de papelera) y, con la app abierta en la primera pantalla, **tocá el botón para ir a la segunda**. Leé la secuencia con atención:

```
Main → onPause
Segunda → onCreate
Segunda → onStart
Segunda → onResume
Main → onStop
```

Pará en esto un minuto, porque es más fino de lo que parece: la primera pantalla se **pausa** antes de que la segunda exista — pero no se **detiene** (`onStop`) hasta que la segunda ya está en escena, interactiva. El sistema nunca deja un hueco: siempre hay una pantalla lista antes de retirar del todo la anterior. Ahora **volvé atrás** y leé:

```
Segunda → onPause
Main → onStart
Main → onResume
Segunda → onStop
Segunda → onDestroy
```

La primera *revive* (no se recrea: no hay `onCreate` — estaba detenida, no muerta), y la segunda, desapilada, **se destruye**: en la pila de retroceso, salir es morir. Todo lo que la teoría contó de la pila, acabás de leerlo en la consola.

✓ *Verificación: las dos secuencias, ida y vuelta, coinciden con lo descrito (el orden exacto de las últimas líneas puede variar levemente entre versiones de Android — lo estructural es: pausa del que se va antes, resume del que llega, stop/destroy después).*

Etapa 3 — La tortura metódica

Ahora que las pantallas hablan, toca hacerles pasar un mal rato — con método científico: primero *predecís*, después *mirás*.

3.1 — La tabla del experimentador. Copiá esta tabla en un papel o archivo aparte. La regla del juego es estricta y es lo que separa el experimento del chapoteo: **escribí tu predicción ANTES de ejecutar cada acción** — qué avisos creés que van a aparecer en Logcat, y de qué pantalla. Después ejecutá, mirá, y anotá lo observado. Una predicción errada vale más que una acertada: es el momento exacto en que tu modelo mental se corrige.

#	Acción (con la Segunda pantalla abierta)	Mi predicción	Lo observado
1	Apretar Home (la app queda en fondo)		
2	Volver a la app desde recientes		
3	Apagar la pantalla con el botón de encendido		
4	Encenderla y desbloquear		

#	Acción (con la Segunda pantalla abierta)	Mi predicción	Lo observado
5	Abrir la cortina de notificaciones completa y cerrarla		
6	Recibir algo encima (abrí otra app desde una notificación, o el diálogo de volumen largo) y volver		

3.2 — Ejecutá los seis, uno por uno. Limpiá el Logcat entre acción y acción (papelera) para leer limpio.

✓ Verificación: tabla completa. Contrastá contra esto recién al final — las respuestas esperables: (1) `onPause` → `onStop`: al fondo se va detenida, no destruida — el proceso sigue vivo, la pantalla no. (2) `onStart` → `onResume`, sin `onCreate`: revivir no es renacer. (3-4) el mismo par `stop/start` que Home — para tu Activity, pantalla apagada y fondo son casi lo mismo: no se la ve. (5) probablemente nada, o solo pausa/resume según versión: la cortina no siempre cuenta como taparte. (6) pausa → stop → start → resume: el ciclo completo de ser tapado y descubierto.

3.3 — La lectura de conjunto. Antes de seguir, tres frases que la tabla acaba de demostrarte y que conviene fijar por escrito en tu registro del laboratorio: **detenerse no es morir** (ninguna de las seis acciones produjo un `onDestroy`); **el sistema avisa siempre, y siempre en orden**; y **tu código no decidió nada de lo que pasó** — el usuario y el sistema movieron todos los hilos, tu Activity solo fue notificada. Eso es, con todas las letras, programar con el control prestado.



Problemas frecuentes de la Etapa 3

- **Me aparecen logs duplicados o secuencias raras:** ¿tenés la app abierta en dos lugares (emulador y teléfono a la vez)? Logcat muestra el dispositivo seleccionado arriba — verificá cuál estás mirando.
- **La acción 5 me dio distinto que a un compañero:** bien — es el punto. El comportamiento fino varía entre versiones y fabricantes; lo estructural (pares pause/resume, stop/start) no. Anotá la diferencia: acabás de tocar la fragmentación.
- **No veo nada en Logcat:** el filtro `tag:VIDA` tiene que estar exacto, y la app tiene que ser la última versión ejecutada desde Android Studio (si la abriste desde el ícono sin re-ejecutar, los logs salen igual; si no aparecen, dale Play de nuevo).

Etapa 4 — El giro fatal

Todo lo anterior fue vida y pausa. Ahora viene la primera muerte — y se lleva algo tuyo.

4.1 — Dale estado a la sala de experimentos. La segunda pantalla necesita algo que doler: agregale al layout (`activity_segunda.xml`, debajo del TextView de bienvenida) un contador y un campo de texto:

```
<TextView
    android:id="@+id/tvContador"
```

```

        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:textSize="48sp"
        android:layout_marginTop="24dp" />

        <Button
            android:id="@+id/btnSumar"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="@string/btn_sumar" />

        <EditText
            android:id="@+id/etNotas"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="24dp"
            android:hint="@string/hint_notas"
            android:inputType="textMultiLine" />

```

Strings: `<string name="btn_sumar">Sumar</string>` y `<string name="hint_notas">Notas del experimento...</string>`. Y en `SegundaActivity.kt`, el estado y su lógica (dentro de la clase; el contenido de `onCreate` se amplía):

```

private var contador = 0

override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    Log.d("VIDA", "Segunda → onCreate")
    setContentView(R.layout.activity_segunda)

    val nombre = intent.getStringExtra("nombre") ?: "misterioso visitante"
    findViewById<TextView>(R.id.tvBienvenida).text =
        "Sala de experimentos de $nombre"

    val tvContador = findViewById<TextView>(R.id.tvContador)
    tvContador.text = contador.toString()

    findViewById<Button>(R.id.btnSumar).setOnClickListener {
        contador++
        tvContador.text = contador.toString()
    }
}

```

Ejecutá, andá a la segunda pantalla, **sumá hasta 7 y escribí una frase** en las notas.

✓ *Verificación: contador en 7, tu frase escrita en el campo.*

4.2 — Girá. Con todo eso en pantalla, **rotá el dispositivo** (en el emulador: los botones de rotación de su barra lateral; si la pantalla no gira, activá la rotación automática en el Android emulado, arrastrando su barra de estado). Mirá primero el **Logcat**:

```

Segunda → onPause
Segunda → onStop
Segunda → onDestroy
Segunda → onCreate
Segunda → onStart
Segunda → onResume

```

Ahí está, en seis líneas: **el sistema destruyó tu Activity y creó otra** — mismo nombre de clase, objeto nuevo, vida nueva. Ahora mirá la pantalla: **el contador volvió a 0**. Tu `contador` era una variable de la instancia destruida; la instancia nueva nació con su valor inicial. El giro se llevó el 7.

4.3 — El detalle que hace todo más interesante. Mirá el campo de notas: **tu frase sigue ahí**. ¿Cómo — si la Activity murió? Porque el sistema, antes de destruir por cambio de configuración, guarda automáticamente el estado de las **vistas que tienen id** (por eso insistimos con los `android:id`), y se lo repone a la instancia nueva. El campo de texto se salvó *porque es una vista con id*; el contador se perdió *porque es una variable tuya* — y de tus variables el sistema no sabe nada. La frontera quedó dibujada con precisión: **el sistema restaura lo suyo; lo tuyo es problema tuyo**.

✓ *Verificación: contador en 0, frase intacta, y la secuencia `destroy→create` leída en el Logcat. Escribí en tu registro la explicación en una línea: ¿por qué el sistema destruye la Activity al rotar, en lugar de simplemente acomodarla? (La respuesta está en la teoría: la configuración cambió — orientación, y con ella posiblemente el layout, los recursos — y recrear es la manera de recargar todo coherente.)*

Problemas frecuentes de la Etapa 4

- **El emulador no rota:** además del botón de rotación del panel del emulador, el Android *de adentro* tiene que tener la rotación automática activada (bajá su barra de estado y fijate el ícono de rotación). Son dos llaves distintas y confunde a todo el mundo.
- **El contador no vuelve a 0:** ¿estás seguro de que giró? En algunos dispositivos el modo escritorio o la rotación bloqueada evitan la recreación. Verificá contra el Logcat: si no hubo `onDestroy`, no hubo recreación.
- **Mi frase también se perdió:** revisá que el `EditText` tenga su `android:id` — sin id, el sistema no puede asociarle el estado guardado.

Etapa 5 — La muerte por memoria, provocada

El giro fue una muerte anunciada: la provocaste vos, mirando. La muerte por memoria es otra cosa — llega cuando no estás, sin aviso, y el usuario ni se entera de que su app dejó de existir. Hoy vas a jugar a ser el sistema.

5.1 — Identificá el proceso. Con la app abierta en la segunda pantalla (contador en algún número, frase escrita), mirá la barra superior del **Logcat**: al lado del nombre del paquete aparece un número — el **PID** (identificador de proceso). Anotalo. Ese número es *esta* encarnación de tu app: si el proceso muere y renace, el número cambia.

✓ *Verificación: PID anotado (ej.: `com.example.holayo (12345)`).*

5.2 — Mandala al fondo y matala. Apretá **Home** (la app queda detenida, proceso vivo — Etapa 3, acción 1). Ahora, el golpe. Abrí la terminal integrada de Android Studio (pestaña **Terminal**, abajo) y ejecutá:

```
adb shell am kill com.example.holayo
```

`adb` es el puente de comandos con el dispositivo; `am kill` le pide al sistema que mate el proceso de esa aplicación **como lo haría por presión de memoria** — solo funciona si la app está en fondo, exactamente como en la vida real: al que está en primer plano no lo toca nadie.

✓ *Verificación: el comando ejecuta sin error. En Logcat, el PID de tu app desaparece del selector o figura como terminado. Tu aplicación, en este momento, no existe: no hay proceso, no hay Activities, no hay variables. Solo queda lo que el sistema haya guardado.*

5.3 — El regreso del fantasma. Volvé a la app **desde los recientes** (no desde el ícono). Mirá todo, en este orden:

- **El Logcat:** la app renace con **PID nuevo** — y la secuencia muestra la recreación de la Activity que estaba arriba de la pila: `Segunda` → `onCreate` → `onStart` → `onResume`. El sistema recordó *dónde estabas* — te devolvió a la sala de experimentos, no al inicio.
- **La pantalla:** la frase del campo de notas **está** (el estado de las vistas con id se guardó antes de morir, igual que en el giro). El contador, **en cero** — tu variable murió con el proceso, otra vez.
- **El saludo:** dice tu nombre — porque el **Intent que creó la pantalla también se conserva y se reentrega**, con su equipaje. El sistema guarda la pila *como receta*: qué Activities, con qué Intents, con qué estado de vistas — y la recocina entera al volver.

✓ *Verificación: PID distinto al anotado + frase viva + contador muerto. Escribí la conclusión en tu registro, porque es LA lección del laboratorio: para el usuario, no pasó nada — apretó Home, volvió, y "la app seguía ahí". Que la continuidad se sienta perfecta mientras por debajo hubo una destrucción total es un logro de ingeniería — del sistema, y del desarrollador que guardó lo que había que guardar. El que no lo hace produce la app que "vuelve en blanco".*

Problemas frecuentes de la Etapa 5

- **adb no se reconoce como comando:** usá la Terminal *de Android Studio* (que ya lo tiene en el camino) y no una terminal externa. Si igual falla, el ejecutable está en la carpeta `platform-tools` del SDK — pero antes probá cerrar y reabrir la pestaña Terminal.
- **El comando corre pero al volver no hubo recreación (mismo PID):** la app no estaba en fondo cuando ejecutaste el kill, o volviste tan rápido que el sistema aún no la había matado. Repetí: Home → comando → esperá 2 segundos → recientes.
- **Al volver me manda a la primera pantalla:** algunos launchers, al abrir *desde el ícono*, resetean la tarea. Por eso el paso dice **desde recientes** — es la diferencia entre "continuar la tarea" y "empezar de nuevo".

Etapa 6 — Mirar por dentro: el depurador

El Logcat te contó la vida de la app como crónica. El depurador es otra cosa: congela el mundo y te deja caminar adentro.

6.1 — Poné el primer breakpoint. En `SegundaActivity.kt`, hacé click en el margen izquierdo del editor, a la altura de la línea `contador++` (dentro del listener del botón). Aparece el círculo rojo: un **breakpoint** — la marca de "cuando la ejecución llegue acá, frená todo".

✓ *Verificación: círculo rojo en el margen de la línea.*

6.2 — Ejecutá en modo depuración. En vez del Play de siempre, tocá el ícono del **bichito** (Debug, o Shift+F9). La app corre normal — hasta que hagas lo que despierta al breakpoint: andá a la segunda

pantalla y **tocá "Sumar"**. La app se congela; Android Studio salta al frente con la ventana **Debug** abierta y la línea resaltada.

✓ *Verificación: la ejecución detenida en `contador++`; el teléfono/emulador, congelado sin responder — el mundo, en pausa.*

6.3 — Caminá adentro. En la ventana Debug, mirá el panel de **variables**: ahí está `contador` con su valor actual, `this` (tu Activity, con todo adentro), los objetos vivos. Ahora avanzá **una línea** con **Step Over** (F8): la línea se ejecuta, y en el panel ves a `contador` cambiar de valor **ante tus ojos**. Seguí un par de pasos, mirá cómo `tvContador.text` toma el valor nuevo. Cuando quieras soltar el mundo: **Resume** (F9) — la app revive como si nada.

✓ *Verificación: viste el valor de `contador` antes y después de la línea, y la app siguió normal tras Resume.*

6.4 — La diferencia, en una línea para tu registro. El Logcat responde "¿qué pasó?" — es la crónica, escrita por quien la previó (vos, al poner cada log). El depurador responde "¿qué está pasando *exactamente* acá?" — sin haber previsto nada, con acceso a todo. El primero es para entender el comportamiento; el segundo, para cazar el error que no entendés. Vas a usar los dos toda la materia — y el criterio de cuál usar cuándo es parte del oficio.



Problemas frecuentes de la Etapa 6

- **El breakpoint no se activa:** ¿ejecutaste con el bichito (Debug) y no con Play? Un breakpoint con ejecución normal es solo un adorno rojo.
- **"Waiting for debugger" eterno al arrancar:** cerrá la app en el dispositivo y volvé a lanzar con Debug desde Android Studio.
- **Todo va lentísimo en modo debug:** es normal — el mundo congelable cuesta. Usalo para cazar, no para vivir.

Desafíos opcionales

Sin guía paso a paso — de eso se trata.

1. **El sexto aviso.** Existe un método del ciclo de vida que este laboratorio no instrumentó: `onRestart`. Investigá cuándo se llama, agregalo a ambas Activities, y encontrá qué experimento de la tabla de la Etapa 3 lo dispara. ¿Por qué la teoría casi no lo menciona?
2. **Compartir con el mundo: el Intent implícito.** El viaje de la Etapa 1 fue explícito — nombraste al destino. Agregale a la primera pantalla un botón "Compartir mi perfil" que arme un texto con tus datos y lo envíe con un Intent **implícito** de acción `ACTION_SEND` (tipo `text/plain`). Al tocarlo, el sistema te va a ofrecer el selector con todas las apps capaces de recibir texto — la colaboración entre aplicaciones de la teoría, en tu pantalla. Pista: `Intent.createChooser`.
3. **Salvar al contador.** Existe un mecanismo para que *tus* variables también sobrevivan a la recreación: `onSaveInstanceState`. Investigalo, guardá el contador, y verificá que sobreviva al giro y a la muerte por memoria de la Etapa 5. Y una pregunta al margen: ahora que lo viste — ¿te imaginás guardando así las veinte variables de una pantalla real? Guardá esa incomodidad.
4. **La tortura del que está abajo.** Toda la tortura fue sobre la pantalla de arriba. Con la segunda en primer plano, ¿qué le puede pasar a la *primera*, que espera abajo en la pila? Diseñá el experimento

(pista: la Etapa 5 mata procesos enteros...) y verificá con los logs qué encuentra el "atrás" cuando el proceso murió en el medio.

☒ Checklist de salida

- ☐ HolaYo tiene dos pantallas conectadas por un Intent explícito que lleva un dato
- ☐ Ambas Activities instrumentadas; sé leer el entrelazado de la navegación en Logcat
- ☐ Tabla de la tortura completa, con predicciones escritas antes de cada observación
- ☐ Presenció la recreación por giro: la secuencia en el log, el contador perdido, la frase sobreviviente — y sé explicar la diferencia
- ☐ Mató el proceso con `adb`, verifiqué el PID nuevo, y sé qué reconstruyó el sistema y qué no
- ☐ Detuve la ejecución con un breakpoint e inspeccioné variables en vivo
- ☐ (Opcional) `onRestart` · Intent implícito de compartir · `onSaveInstanceState` · la tortura del de abajo

La pregunta que este laboratorio deja abierta — a propósito: viste dónde muere el estado: en las variables de una pantalla que el sistema destruye cuando quiere. Viste el remedio artesanal (guardar valor por valor en `onSaveInstanceState`) y, si hiciste el desafío 3, sentiste su incomodidad. Entonces: **¿dónde debería vivir el estado de una pantalla, para que ni el giro ni la memoria se lo lleven — sin guardarlo a mano valor por valor?** La respuesta tiene nombre, tiene teoría (el bloque de los modelos de desarrollo), y tiene laboratorio: es exactamente lo que vas a construir en el **Laboratorio 3**.